

Depuración y transformación del dataset de enfermedades derivadas de la ocupación en trabajadores informales de Bogotá

Proyecto: Sistema de predicción de riesgos con IA

Aprendiz

[Escribe aquí tu nombre completo]

Fecha

28 de septiembre de 2026

Repositorio

github.com/CLaireMor/prediccion-riesgos-ia

Contenido

1. Introducción
 2. Descripción del dataset base
 3. Resumen del proceso de depuración y transformación
 4. Tablas comparativas antes y después
 5. Gráficos de apoyo
 6. Impacto en el modelo de IA
 7. Conclusiones
 8. Verificación de la lista de chequeo
- Anexo A. Código de programación
- Anexo B. Enlaces y fuentes

1. Introducción

Bogotá vigila la salud de los trabajadores de la economía informal a través del Subsistema de Vigilancia Epidemiológica Ocupacional de las y los Trabajadores de la Economía Informal (**SIVISTRA**) de la Secretaría Distrital de Salud. Cada notificación describe al trabajador (edad, sexo, escolaridad, aseguramiento, ingresos), su unidad de trabajo informal (UTI), la ocupación, el agente de riesgo al que se expone y el tipo de lesión o sistema comprometido.

El proyecto usa estos datos para entrenar un modelo de inteligencia artificial que, a partir de la **edad, el sexo, la ocupación y el agente de riesgo**, estima qué tipo de lesión es más probable. El resultado se ofrece en una aplicación web que apoya la prevención: permite anticipar lesiones típicas por oficio, tamizar bases de trabajadores y priorizar intervenciones.

1.1 Justificación técnica del análisis

Un modelo de IA solo puede ser tan bueno como los datos con los que aprende. El dataset original presenta problemas que, sin tratamiento, degradan el modelo: categorías duplicadas por espacios sobrantes, valores sin sentido como "0", registros repetidos, un objetivo con 60 clases de las cuales muchas tienen uno o dos casos, y variables categóricas que la versión inicial del proyecto convertía en números arbitrarios (0, 1, 2...), inventando un orden que no existe.

Este informe documenta cada decisión de limpieza, codificación y escalado, explica por qué se eligió cada técnica según el tipo de dato y el modelo previsto (regresión logística multinomial con regularización), y mide su efecto con tablas y gráficos comparativos.

1.2 Modelo de IA previsto

Se usa una **regresión logística multinomial** con regularización L2. Es un modelo lineal, interpretable y bien calibrado, adecuado para un objetivo categórico con muchas clases. Dos propiedades condicionan la preparación: necesita que las categóricas sin orden entren como variables indicadoras (One-Hot), y su regularización penaliza los coeficientes por igual, por lo que las variables numéricas deben estar en escalas comparables.

2. Descripción del dataset base

2.1 Dimensión

El archivo **obs_salud1.csv** contiene **22.438 registros** (filas) y **14 atributos** (columnas), con separador punto y coma y codificación Latin-1. Cubre notificaciones de 2017–2025 en las 20 localidades de Bogotá. Supera ampliamente el mínimo requerido de 10 atributos y 100 registros.

2.2 Tipología de variables

El dataset combina **2 variables numéricas** (Año y Edad) y **12 categóricas**. Entre las categóricas, 2 tienen un orden explícito (Escolaridad y Nivel de ingresos) y 10 son nominales, sin orden.

Variable	Tipo	Subtipo	Distintos	Rol en el proyecto
Año	numérica	Discreta	9	Contexto temporal
Localidad de ocurrencia	categórica	Nominal	20	Contexto geográfico
Sexo	categórica	Nominal	3	Predictor
Edad	numérica	Discreta	76	Predictor
Régimen de aseguramiento	categórica	Nominal	7	Vulnerabilidad
Escolaridad	categórica	Ordinal	12	Vulnerabilidad
Ocupación	categórica	Nominal	228	Predictor
Tipo UTI	categórica	Nominal	2	Contexto laboral
Clase UTI	categórica	Nominal	2	Contexto laboral
Nivel de ingresos	categórica	Ordinal	5	Vulnerabilidad
Forma de pago	categórica	Nominal	5	Contexto laboral
Síntoma	categórica	Nominal	232	Excluida (posterior a la lesión)
Agente probablemente asociado	categórica	Nominal	76	Predictor
Tipo de lesión (objetivo)	categórica	Nominal	61	Variable objetivo

Tabla 1. Variables del dataset original, su tipo y su rol. "Distintos" = número de valores únicos.

2.3 Origen documentado

Fuente: Secretaría Distrital de Salud de Bogotá, Observatorio de Salud de Bogotá (SaluData), subsistema SIVISTRA. Conjunto de datos "Enfermedades derivadas de la ocupación en Unidades de Trabajo Informal (UTI) en Bogotá D.C.", publicado en el portal de Datos Abiertos Bogotá. Los enlaces completos están en el Anexo B.

Portal de datos abiertos: datosabiertos.bogota.gov.co/dataset/enfermedades-derivadas-de-la-ocupacion-en-unidades-de-trabajo-informal-uti-en-bogota-d-c

Indicador en SaluData:

saludata.saludcapital.gov.co/osb/indicadores/enfermedades-derivadas-de-la-ocupacion

3. Resumen del proceso de depuración y transformación

El proceso completo está en el script **preparacion_datos.py** (Anexo A) y se ejecuta en este orden. Cada paso registra cuántos valores o filas afectó, y esas cifras son las que aparecen en este informe.

3.1 Coherencia de tipos y valores

- **Tipos por variable.** Año y Edad se leen como enteros; las otras 12 como texto. Se verificó que ninguna variable numérica viniera como texto ni al revés.

- **Rangos.** Edad va de 15 a 95 años y Año de 2017 a 2025. No hay valores imposibles (edades negativas o años futuros). Las edades de 15 a 17 años (33 casos) no se eliminan: son reales y señalan posible trabajo adolescente.
- **Espacios sobrantes.** "Especial" y "Especial " aparecían como dos regímenes distintos. Al normalizar espacios, Régimen pasa de 7 → 6 categorías y Síntoma de 232 → 227 categorías. También se limpiaron etiquetas del objetivo como "Afecciones de vía respiratoria baja".
- **Valores sin sentido.** Un "0" en Forma de pago, un "0" en Tipo de lesión y 15 respuestas "NS / NR" en Nivel de ingresos se convirtieron en faltantes, para tratarlos con una regla explícita en lugar de dejarlos como categorías falsas.

3.2 Duplicados

Se encontraron **264 filas idénticas en las 14 variables**, incluidas la edad exacta y el síntoma. Sin un identificador de caso, la explicación más probable es una notificación repetida del mismo caso, así que se eliminaron conservando la primera aparición. Representan el 1,2 % de los registros, por lo que su eliminación no altera la distribución y evita que el modelo cuente dos veces el mismo caso (y que un caso quede a la vez en entrenamiento y en prueba).

3.3 Tratamiento de faltantes

Los faltantes son muy pocos, pero cada uno recibió una decisión justificada según el papel de la variable:

Variable	Faltantes	Decisión	Justificación
Ocupación	6	Eliminar fila	Es predictor principal del modelo; imputarla inventaría una exposición laboral.
Tipo de lesión	1	Eliminar fila	Es la variable objetivo: un registro sin respuesta no sirve para entrenar ni evaluar.
Régimen de aseguramiento	1	Imputar moda ("Contributivo")	Variable secundaria categórica; con tan pocos casos la moda no distorsiona la distribución.
Escolaridad	1	Imputar moda ("Secundaria Completa")	Variable secundaria categórica; con tan pocos casos la moda no distorsiona la distribución.
Clase UTI	1	Imputar moda ("Fija")	Variable secundaria categórica; con tan pocos casos la moda no distorsiona la distribución.
Nivel de ingresos	12	Imputar moda ("1 SMMLV")	Variable secundaria categórica; con tan pocos casos la moda no distorsiona la distribución.
Forma de pago	1	Imputar moda ("A destajo")	Variable secundaria categórica; con tan pocos casos la moda no distorsiona la distribución.

Tabla 2. Faltantes detectados (tras eliminar duplicados) y su tratamiento. En total se eliminaron 7 filas.

Se usó la **moda** y no la media o la mediana porque todas las variables con faltantes son categóricas: la media no existe para ellas. En Nivel de ingresos, aunque es ordinal, la moda ("1 SMMLV") coincide con la mediana, así que ambas opciones darían el mismo resultado.

3.4 Variable objetivo

El tipo de lesión tenía **60 clases**. Muchas tienen menos de 30 casos y, al revisarlas, casi todas son registros con varias lesiones concatenadas en una sola celda (por ejemplo "Artrosis Dorsolumbalgias Otros Trastornos De Tejidos Blandos"). Con tan pocos ejemplos el modelo no puede aprenderlas, así que se agruparon en "Otras lesiones / registros múltiples". Quedan **26 clases** y se reagrupan solo 114 registros (0,5 %).

3.5 Codificación categórica

- **One-Hot Encoding** para las 8 nominales: Sexo, Localidad, Régimen, Ocupación, Tipo UTI, Clase UTI, Forma de pago y Agente. Cada categoría se vuelve una columna 0/1. Las categorías con menos de 10 casos se agrupan en una columna "infrecuente" para no crear cientos de columnas casi vacías; así Ocupación genera 84 columnas en lugar de 228 y Agente 43 en lugar de 76.
- **Ordinal Encoding** para las 2 variables con jerarquía, con el orden definido a mano (no alfabético):
 - Escolaridad (0 a 11): No Fue A La Escuela < Primaria Incompleta < Primaria Completa < Secundaria Incompleta < Secundaria Completa < Técnico Pos Secundaria Incompleto < Técnico Pos Secundaria < Técnico Pos Secundaria Completo < Universidad Incompleta < Universidad Completa < Posgrado Incompleto < Posgrado Completo.
 - Nivel de ingresos (0 a 3): Menos de 1 SMMLV < 1 SMMLV < Entre 1 y 2 SMMLV < 2 y MÁS SMMLV.
- **Síntoma** se excluye de la matriz: se registra cuando la lesión ya apareció, así que usarlo para predecirla sería fuga de información y el modelo no serviría para prevenir.
- **Objetivo:** las 26 clases se codifican como etiquetas 0-25, que es lo que espera el clasificador (no son variables de entrada, así que el número no introduce orden).

3.6 Escalado numérico

- **Z-score** (media 0, desviación estándar 1) para **Edad**. Su distribución es aproximadamente simétrica y tiene algunos valores altos (hasta 95 años). El Z-score no comprime el resto de los datos por culpa de esos extremos, algo que sí hace Min-Max.
- **Min-Max** (0 a 1) para **Año** y para los códigos ordinales de **Escolaridad** e **Ingresos**. Son variables acotadas y sin atípicos, y Min-Max conserva el orden y los deja en el mismo rango que las columnas One-Hot (0/1).

3.7 Justificación técnica de cada técnica

Técnica	Aplicada a	Por qué	Alternativa descartada
Normalizar texto	Todas las categóricas	Evita categorías falsas por espacios.	Dejarlas: duplica categorías.
Eliminar duplicados	264 filas	Evita contar dos veces un caso y la fuga entre entrenamiento y prueba.	Conservarlos: sesga frecuencias.

Técnica	Aplicada a	Por qué	Alternativa descartada
Eliminar filas	Ocupación, Lesión	Variable clave u objetivo: imputar inventaría información.	Imputar moda: crea exposiciones falsas.
Imputar moda	5 variables secundarias	Catóricas con 1-12 faltantes; no distorsiona.	Media/mediana: no aplican a categorías.
One-Hot	8 nominales	Sin orden natural; la regresión logística necesita indicadoras.	LabelEncoder: inventa un orden $0 < 1 < 2$.
Ordinal	Escolaridad, Ingresos	Tienen jerarquía explícita que conviene conservar.	One-Hot: pierde el orden.
Z-score	Edad	Simétrica, con atípicos; la regularización L2 exige escalas comparables.	Min-Max: comprime por los extremos.
Min-Max	Año, ordinales	Acotadas, sin atípicos; quedan en 0-1 como las One-Hot.	Z-score: igual de válido, menos interpretable.

Tabla 3. Resumen de técnicas, a qué se aplicaron y por qué.

4. Tablas comparativas antes y después

4.1 Estructura general

Indicador	Antes	Después de depurar	Después de transformar
Registros (filas)	22.438	22.167	22.167
Atributos (columnas)	14	14	167 + objetivo
Valores faltantes o sin sentido	26	0	0
Filas duplicadas	264	0	0
Categorías de Régimen	7	6	6 columnas 0/1
Clases del objetivo	61	26	26 etiquetas (0-25)
Tipos de dato	2 numéricas + 12 texto	2 numéricas + 12 texto	167 numéricas

Tabla 4. Estructura del dataset en cada etapa. En "Antes", el objetivo cuenta también el valor "0"; los faltantes incluyen "0" y "NS / NR" y se cuentan antes de eliminar duplicados.

4.2 Por variable

Variable	Faltantes antes	Distintos antes	Faltantes después	Distintos después	Tratamiento
Año	0	9	0	9	Min-Max (0-1)
Localidad de ocurrencia	0	20	0	20	One-Hot
Sexo	0	3	0	3	One-Hot
Edad	0	76	0	76	Z-score
Régimen de aseguramiento	1	7	0	6	Limpieza + One-Hot
Escolaridad	1	12	0	12	Ordinal + Min-Max
Ocupación	6	228	0	228	One-Hot (mín. 10 casos)
Tipo UTI	0	2	0	2	One-Hot
Clase UTI	1	2	0	2	One-Hot
Nivel de ingresos	15	5	0	4	Ordinal + Min-Max

Variable	Faltantes antes	Distintos antes	Faltantes después	Distintos después	Tratamiento
Forma de pago	1	5	0	4	One-Hot
Síntoma	0	232	0	227	No se usa
Agente probablemente asociado	0	76	0	76	One-Hot (mín. 10 casos)
Tipo de lesión (objetivo)	1	61	0	60	Agrupación + codificación de etiqueta

Tabla 5. Comparación por variable. Los faltantes "antes" incluyen valores sin sentido ("0", "NS / NR").

4.3 Estadísticos de las variables escaladas

Variable	Media	Desv. estándar	Mínimo	Máximo
Edad (original)	44,587	13,777	15,000	95,000
Edad (Z-score)	0,000	1,000	-2,148	3,659
Año (original)	2021,178	2,096	2017,000	2025,000
Año (Min-Max)	0,522	0,262	0,000	1,000
Escolaridad (ordinal 0-1)	0,372	0,185	0,000	1,000
Ingresos (ordinal 0-1)	0,376	0,246	0,000	1,000

Tabla 6. El Z-score deja la Edad con media 0 y desviación 1; Min-Max deja Año y los ordinales entre 0 y 1.

4.4 Ejemplo de un registro antes y después

Campo original	Valor original	Columnas resultantes
Edad	63	zscore__Edad = 1,337
Sexo	Femenino	onehot__Sexo_Femenino = 1; demás columnas de Sexo = 0
Escolaridad	Secundaria Incompleta	ordinal__Escolaridad = 3/11 = 0,273
Nivel de ingresos	1 SMMLV	ordinal__Ingresos = 1/3 = 0,333
Año	2019	minmax__Anio = (2019 - 2017)/8 = 0,250
Ocupación	Zapateros Y Afines	onehot__Ocupacion_Zapateros Y Afines = 1; demás = 0

Tabla 7. Cómo se transforma el primer registro del dataset (una zapatera de 63 años de Tunjuelito, 2019).

5. Gráficos de apoyo

Los gráficos muestran el estado de los datos antes y después de cada transformación.

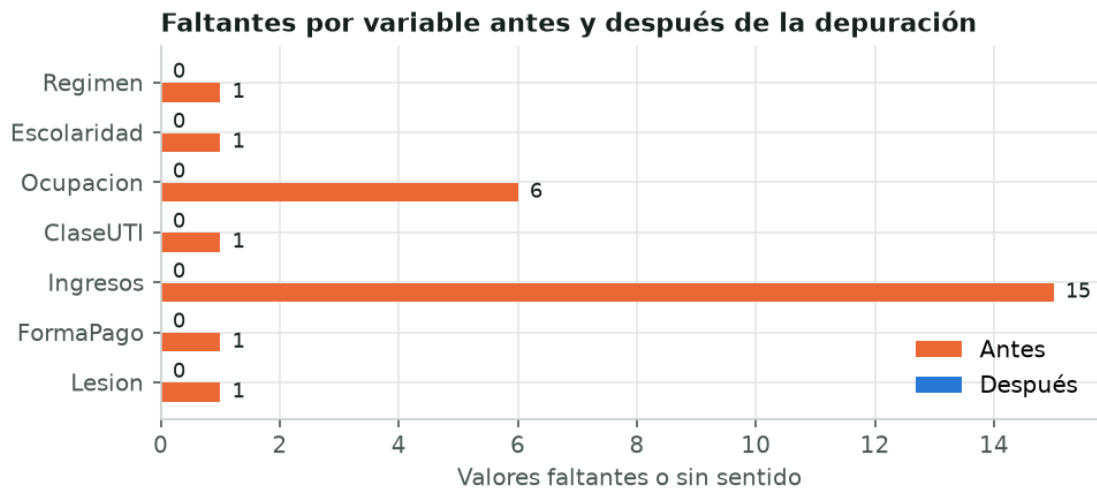


Figura 1. Valores faltantes o sin sentido por variable. Después de la depuración no queda ninguno.

Histogramas: el escalado cambia la escala, no la forma de la distribución

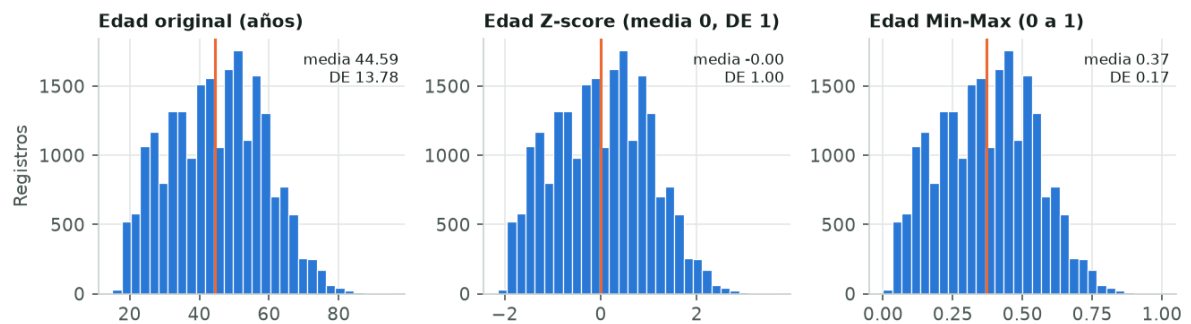


Figura 2. Histogramas de Edad original, con Z-score y con Min-Max. La forma de la distribución es idéntica; solo cambia la escala. Con Z-score la media pasa de 44,6 años a 0 y la desviación de 13,8 a 1.

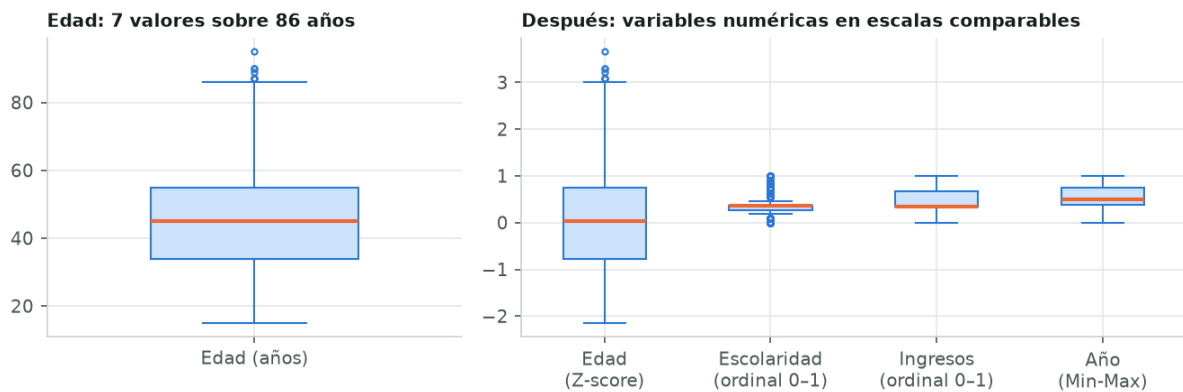


Figura 3. Boxplots. A la izquierda, Edad en años: 7 valores superan el límite de 86,5 años ($Q3 + 1,5 \times IQR$). Se conservan porque son edades reales. A la derecha, las variables ya escaladas quedan en rangos comparables.

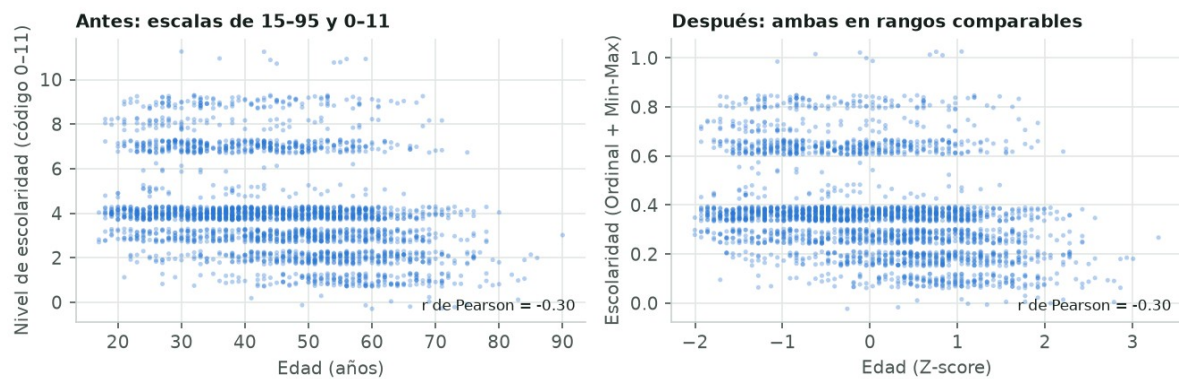
Dispersión Edad vs. Escolaridad (muestra de 3.000; la relación se conserva)

Figura 4. Dispersión de Edad frente a Escolaridad, antes y después (muestra aleatoria de 3.000 registros). La correlación ($r = -0,302$: a mayor edad, menor escolaridad) se conserva exactamente, porque ambas transformaciones son lineales y respetan el orden.

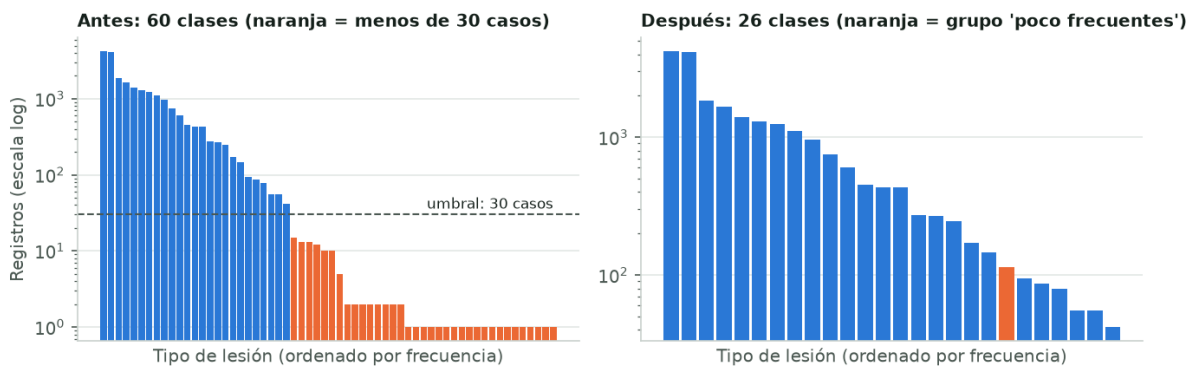


Figura 5. Distribución del objetivo en escala logarítmica. Antes: 60 clases, muchas con 1 o 2 casos. Después: 26 clases, todas con al menos 30 casos.

6. Impacto en el modelo de IA

Para medir si la preparación mejora el modelo, se entrenaron dos versiones con la misma partición 80/20 y se compararon con una línea base que predice siempre la lesión más común, sin mirar al trabajador:

Tratamiento	Acierto 1ª opción	Acierto top 3	Log-loss (menor es mejor)
Original: LabelEncoder + MLP (sin depurar)	36,4 %	72,0 %	1,88
Depurado: One-Hot + Z-score + Regresión logística	43,1 %	78,3 %	1,63
Línea base: predecir la prevalencia	18,9 %	46,0 %	2,63

Tabla 8. Desempeño con la misma partición de prueba. "Top 3" = la lesión real está entre las 3 que el modelo considera más probables.

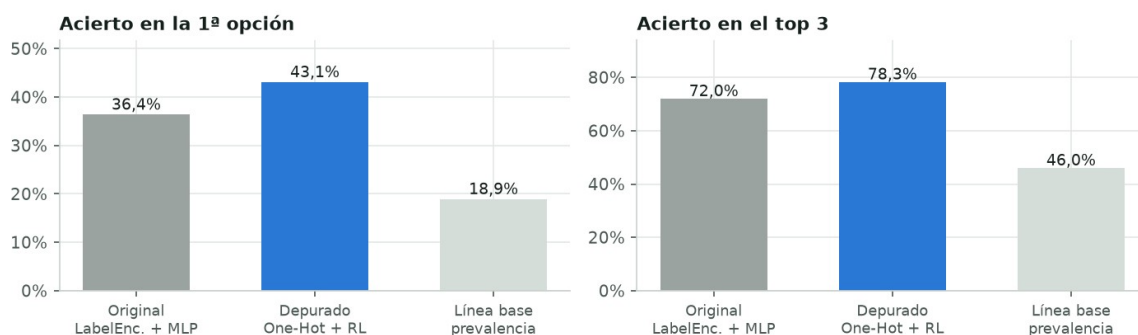


Figura 6. Comparación del acierto entre el tratamiento original, el depurado y la línea base.

El tratamiento depurado sube el acierto en la primera opción de **36,4 % a 43,1 %** y en el top 3 de **72,0 % a 78,3 %**, además de reducir el log-loss (mejor calibración). La mejora no viene de un modelo más complejo, sino de lo contrario: se pasó de una red neuronal a una regresión logística, más simple. El cambio que importa es la preparación, sobre todo One-Hot en lugar de números arbitrarios y la limpieza del objetivo.

Esta cifra es optimista porque mezcla años. Si se entrena con datos anteriores a 2024 y se prueba con 2024-2025, que es como se usaría en la práctica, el modelo acierta la primera opción en cerca del **29 %** y el top 3 en cerca del **66 %**, frente a 18 % y 42 % de la línea base. La caída se debe a que la forma de registrar los casos cambia entre años (en 2023, el 44 % de los casos se codificó como "Otros trastornos de tejidos blandos").

7. Conclusiones

- **La calidad de los datos pesa más que la complejidad del modelo.** Con los mismos datos de origen, una preparación correcta y un modelo más simple superan a la red neuronal original: +6,7 puntos en la primera opción y +6,3 en el top 3.
- **Codificar según el tipo de dato es decisivo.** Tratar Ocupación o Agente como números (0, 1, 2...) le dice al modelo que "peluquero < mecánico", y eso es falso. One-Hot elimina ese sesgo, y la codificación ordinal conserva la jerarquía real de Escolaridad e Ingresos.
- **La limpieza fue pequeña en volumen pero importante en efecto.** Se eliminó solo el 1,2 % de los registros (271 filas), pero quitar duplicados, categorías falsas y valores sin sentido evita que el modelo aprenda artefactos del registro.
- **El escalado no cambia la información, cambia la escala.** Los histogramas y la dispersión lo confirman: la forma de las distribuciones y las correlaciones se conservan, y todas las variables quedan en rangos comparables. Eso es lo que necesita la regularización L2 para tratarlas con equidad.
- **Aplicabilidad en IA.** El dataset transformado (22.167×167 variables numéricas) queda listo para regresión logística y sirve también para otros modelos lineales, redes neuronales o máquinas de vectores de soporte. Los

modelos de árboles no necesitarían el escalado, pero sí se benefician de la limpieza.

- **Límites.** Los datos contienen solo casos notificados: el modelo estima qué lesión es más probable si el trabajador llega a ser un caso, no la probabilidad de lesionarse. Además, el agente de riesgo es el "probablemente asociado" según quien registra. Por eso los resultados apoyan la prevención, pero no reemplazan una valoración clínica.

8. Verificación de la lista de chequeo

Requisito	Dónde se cumple	Estado
Dimensión mínima (≥ 10 atributos, ≥ 100 registros)	Sección 2.1: 14 atributos, 22.438 registros	Cumple
Tipología mixta	Sección 2.2, Tabla 1: 2 numéricas y 12 categóricas	Cumple
Origen documentado	Sección 2.3 y Anexo B	Cumple
Tratamiento de nulos/faltantes	Sección 3.3, Tabla 2: moda y eliminación justificada	Cumple
Coherencia y duplicados	Secciones 3.1 y 3.2: tipos, rangos, texto y 264 duplicados	Cumple
Codificación categórica	Sección 3.5: One-Hot (8 nominales) y Ordinal (2 ordinales)	Cumple
Escalado numérico	Sección 3.6: Z-score (Edad) y Min-Max (Año, ordinales)	Cumple
Justificación técnica	Sección 3.7, Tabla 3	Cumple
Tablas antes y después	Sección 4, Tablas 4 a 7	Cumple
Histogramas, dispersión y boxplots	Sección 5, Figuras 2, 3 y 4	Cumple
Portada (título, aprendiz, fecha)	Portada	Falta el nombre
Introducción	Sección 1	Cumple
Resumen del proceso	Sección 3	Cumple
Tablas y gráficos integrados	Secciones 4, 5 y 6	Cumple
Conclusiones	Sección 7	Cumple
Anexos: código y enlaces	Anexos A y B	Cumple

Tabla 9. Cada requisito de la lista de chequeo y la sección donde se cumple.

Anexo A. Código de programación

Todo el código está en Python 3.11 con pandas, scikit-learn y matplotlib, y se encuentra en el repositorio del proyecto. Para reproducir los resultados de este informe:

```
pip install -r requirements.txt matplotlib
python preparacion_datos.py      # depuración, transformación, tablas y figuras
python -m pytest tests           # pruebas automáticas del modelo
```

A.1 preparacion_datos.py (depuración, codificación, escalado y gráficos)

```
"""Depuración y transformación del dataset, con evidencia para el informe.
```

```
Genera en informe/:
```

- figuras/*.png gráficos antes/después
- resultados.json tablas y cifras que cita el informe

```
y en salidas/ (no se versiona):
```

- obs_salud_limpio.csv dataset depurado (legible)
- obs_salud_transformado.csv matriz lista para el modelo (codificada y escalada)

```
Uso: python preparacion_datos.py
```

```
"""
```

```
import json
```

```
from pathlib import Path
```

```
import matplotlib
```

```
matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, log_loss, top_k_accuracy_score
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import (
```

```
    LabelEncoder,
```

```
    MinMaxScaler,
```

```
    OneHotEncoder,
```

```
    OrdinalEncoder,
```

```
    StandardScaler,
```

```
)
```

```
import modelo as m
```

```
RAIZ = Path(__file__).parent
```

```
FIG = RAIZ / "informe" / "figuras"
```

```
SALIDAS = RAIZ / "salidas"
```

```
# Orden explícito de las variables con jerarquía
```

```
ORDEN_ESCOLARIDAD = [
```

```
    "No Fue A La Escuela",
```

```
    "Primaria Incompleta",
```

```
    "Primaria Completa",
```

```
    "Secundaria Incompleta",
```

```
    "Secundaria Completa",
```

```

    "Técnico Pos Secundaria Incompleto",
    "Técnico Pos Secundaria", # sin especificar si terminó: se ubica entre incompleto y completo
    "Técnico Pos Secundaria Completo",
    "Universidad Incompleta",
    "Universidad Completa",
    "Posgrado Incompleto",
    "Posgrado Completo",
]
ORDEN_INGRESOS = ["Menos de 1 SMMLV", "1 SMMLV", "Entre 1 y 2 SMMLV", "2 y MÁS SMMLV"]

NOMINALES = ["Sexo", "Localidad", "Regimen", "Ocupacion", "TipoUTI", "ClaseUTI", "FormaPago", "Agente"]
ORDINALES = ["Escolaridad", "Ingresos"]
NUM_Z = ["Edad"]
NUM_MINMAX = ["Anio"]

AZUL, NARANJA, GRIS, TINTA, TINTA2 = "#2a78d6", "#eb6834", "#9aa3a0", "#15201b", "#4b5852"
plt.rcParams.update({
    "font.family": "DejaVu Sans", "font.size": 9.5, "axes.edgecolor": "#c9d1cd",
    "axes.labelcolor": TINTA2, "xtick.color": TINTA2, "ytick.color": TINTA2,
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
    "grid.color": "#e3e8e5", "grid.linewidth": 0.8, "axes.axisbelow": True,
    "axes.titleweight": "bold", "axes.titlesize": 10.5, "axes.titlecolor": TINTA,
    "figure.dpi": 150, "savefig.bbox": "tight",
})

def estructura(df: pd.DataFrame) -> list[dict]:
    """Resumen por variable: tipo, nulos, valores distintos y ejemplo."""
    filas = []
    for c in df.columns:
        s = df[c]
        filas.append({
            "Variable": c,
            "Tipo": "numérica" if pd.api.types.is_numeric_dtype(s) else "categórica",
            "dtype": str(s.dtype),
            "Nulos": int(s.isna().sum()),
            "Distintos": int(s.nunique(dropna=True)),
            "Ejemplo": str(s.dropna().iloc[0])[40],
        })
    return filas

def depurar(crudo: pd.DataFrame, log: list) -> pd.DataFrame:
    df = crudo.copy()
    df.columns = [c.strip() for c in df.columns]
    df = df.rename(columns=m.COLUMNAS)

    # 1. Coherencia de texto: espacios sobrantes crean categorías falsas ("Especial" vs "Especial ")
    antes = {c: df[c].nunique() for c in df.columns if df[c].dtype == object or
str(df[c].dtype).startswith("str")}
    for c in antes:
        df[c] = df[c].astype("string").str.strip().str.replace(r"\s+", " ", regex=True)
    cambios = {c: (antes[c], int(df[c].nunique())) for c in antes if antes[c] != df[c].nunique()}
    log.append({"paso": "Normalizar espacios en textos", "detalle": {k: f"{a} → {b}" for k, (a,
b) in cambios.items()}})

    # 2. Valores incoherentes -> faltantes
    marcadores = {}
    for c in df.columns:
        if c in ("Anio", "Edad"):
            continue

```

```

        n = int(df[c].isin(["0", ""]).sum())
        if n:
            marcadores[c] = n
            df.loc[df[c].isin(["0", ""]), c] = pd.NA
    n_nsnr = int((df["Ingresos"] == "NS / NR").sum())
    df.loc[df["Ingresos"] == "NS / NR", "Ingresos"] = pd.NA
    marcadores["Ingresos (NS / NR)"] = n_nsnr
    log.append({"paso": "Valores sin sentido ('0', 'NS / NR') marcados como faltantes", "detalle":
marcadores})

# 3. Rango de las numéricas
rango = {"Edad": [int(df["Edad"].min()), int(df["Edad"].max())], "Anio": [int(df["Anio"].min()),
int(df["Anio"].max())]}
    log.append({"paso": "Rangos numéricos verificados (sin valores imposibles)", "detalle": rango})

# 4. Duplicados exactos
n_dup = int(df.duplicated().sum())
    df = df.drop_duplicates().reset_index(drop=True)
    log.append({"paso": "Eliminar duplicados exactos (14 variables idénticas)", "detalle": {"filas
eliminadas": n_dup}})

# 5. Faltantes: eliminar si es variable clave u objetivo, moda si es secundaria
nulos = df.isna().sum()
    nulos = {c: int(v) for c, v in nulos.items() if v}
    clave = [c for c in ["Ocupacion", "Agente", "Sexo", "Edad", "Lesion"] if c in nulos]
    n_antes = len(df)
    df = df.dropna(subset=clave).reset_index(drop=True)
    imputadas = {}
    for c in df.columns:
        if df[c].isna().any():
            moda = df[c].mode().iloc[0]
            imputadas[c] = {"n": int(df[c].isna().sum()), "moda": str(moda)}
            df[c] = df[c].fillna(moda)
    log.append({
        "paso": "Tratar faltantes",
        "detalle": {"nulos detectados": nulos, "filas eliminadas (variable clave u objetivo)": n_antes -
len(df),
                    "imputación por moda": imputadas},
    })

# 6. Objetivo: agrupar lesiones con < 30 casos (en su mayoría registros con varias lesiones pegadas)
vc = df["Lesion"].value_counts()
    raras = vc[vc < m.MIN_CASOS_LESION].index
    df["Lesion_Modelo"] = df["Lesion"].where(~df["Lesion"].isin(raras), m.ETIQUETA_OTRAS)
    log.append({"paso": "Agrupar clases raras del objetivo", "detalle": {
        "clases antes": int(vc.size), "clases después": int(df["Lesion_Modelo"].nunique()),
        "registros agrupados": int(df["Lesion"].isin(raras).sum())}})

df["Anio"] = df["Anio"].astype(int)
df["Edad"] = df["Edad"].astype(int)
return df

def transformador() -> ColumnTransformer:
    return ColumnTransformer([
        ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exist",
min_frequency=m.MIN_FRECUENCIA_CATEGORIA,
                                sparse_output=False), NOMINALES),
        ("ordinal", Pipeline([
            ("codificar", OrdinalEncoder(categories=[ORDEN_ESCOLARIDAD, ORDEN_INGRESOS])),
            ("escalar", MinMaxScaler()),

```

```

    ]), ORDINALES),
    ("zscore", StandardScaler(), NUM_Z),
    ("minmax", MinMaxScaler(), NUM_MINMAX),
], verbose_feature_names_out=True)

def metricas(y_te, P, clases):
    return {"top1": accuracy_score(y_te, clases[P.argmax(1)]),
            "top3": top_k_accuracy_score(y_te, P, k=3, labels=clases),
            "log_loss": log_loss(y_te, P, labels=clases)}

def impacto_en_modelo(crudo: pd.DataFrame, limpio: pd.DataFrame) -> list[dict]:
    """Compara el tratamiento original (LabelEncoder + MLP) con el nuevo, misma partición."""
    res = []
    # Original: textos sin normalizar, LabelEncoder en todo, 61 clases
    df0 = crudo.copy()
    df0.columns = [c.strip() for c in df0.columns]
    df0 = df0.rename(columns=m.COLUMNAS)[["Lesion", "Agente", "Ocupacion", "Sexo", "Edad"]].dropna()
    X0 = pd.DataFrame({"Edad": df0["Edad"]})
    for c in ["Sexo", "Agente", "Ocupacion"]:
        X0[c] = LabelEncoder().fit_transform(df0[c])
    y0 = df0["Lesion"].str.strip()
    tr, te = train_test_split(df0.index, test_size=0.2, random_state=42)
    p0 = Pipeline([("s", StandardScaler()), ("mlp", MLPClassifier((32, 16), max_iter=300,
random_state=42))])
    p0.fit(X0.loc[tr], y0[tr])
    mask = y0[te].isin(p0.classes_)
    res.append({"Tratamiento": "Original: LabelEncoder + MLP (sin depurar)",
                **metricas(y0[te][mask], p0.predict_proba(X0.loc[te][mask]), p0.classes_)})

    # Nuevo: dataset depurado + one-hot + z-score + regresión logística
    y = limpio["Lesion_Modelo"]
    tr, te = train_test_split(limpio.index, test_size=0.2, random_state=42, stratify=y)
    p1 = m.construir_pipeline().fit(limpio.loc[tr, m.FEATURES], y[tr])
    res.append({"Tratamiento": "Depurado: One-Hot + Z-score + Regresión logística",
                **metricas(y[te], p1.predict_proba(limpio.loc[te, m.FEATURES]), p1.classes_)})
    prior = y[tr].value_counts(normalize=True).sort_index()
    res.append({"Tratamiento": "Línea base: predecir la prevalencia",
                **metricas(y[te], np.tile(prior.to_numpy(), (len(te), 1)), prior.index.to_numpy())})
    return [{k: (round(v, 4) if isinstance(v, float) else v) for k, v in r.items()} for r in res]

# ----- Gráficos -----
def fig_nulos(crudo_ren, limpio):
    cols = [c for c in crudo_ren.columns if crudo_ren[c].isna().any() or
(crudo_ren[c].astype(str).str.strip().isin(["0", "NS / NR"])).any()]
    antes = [int(crudo_ren[c].isna().sum() + crudo_ren[c].astype(str).str.strip().isin(["0", "NS /
NR"]).sum()) for c in cols]
    despues = [int(limpio[c].isna().sum()) for c in cols]
    fig, ax = plt.subplots(figsize=(7, 2.8))
    y = np.arange(len(cols))
    ax.barh(y + 0.2, antes, 0.38, color=NARANJA, label="Antes")
    ax.barh(y - 0.2, despues, 0.38, color=AZUL, label="Después")
    for i, v in enumerate(antes):
        ax.text(v + 0.2, i + 0.2, str(v), va="center", fontsize=8.5, color=TINTA)
    for i, v in enumerate(despues):
        ax.text(v + 0.2, i - 0.2, str(v), va="center", fontsize=8.5, color=TINTA)
    ax.set_yticks(y, cols)
    ax.invert_yaxis()
    ax.set_xlabel("Valores faltantes o sin sentido")

```



```

ax.set_title("Faltantes por variable antes y después de la depuración", loc="left")
ax.legend(frameon=False, loc="lower right")
fig.savefig(FIG / "01_nulos.png")
plt.close(fig)

def fig_histogramas(limpio, Xt):
    fig, axs = plt.subplots(1, 3, figsize=(9, 2.9))
    datos = [(limpio["Edad"], "Edad original (años)"), (Xt["zscore__Edad"], "Edad Z-score (media 0, DE
1)"),
              (MinMaxScaler().fit_transform(limpio[["Edad"]]).ravel(), "Edad Min-Max (0 a 1)")]
    for ax, (d, t) in zip(axs, datos):
        ax.hist(d, bins=30, color=AZUL, edgecolor="white", linewidth=0.6)
        ax.set_title(t, loc="left", fontsize=9.5)
        ax.axvline(np.mean(d), color=NARANJA, lw=1.5)
        ax.text(0.98, 0.95, f"media {np.mean(d):.2f}\nDE {np.std(d):.2f}", transform=ax.transAxes,
ha="right", va="top", fontsize=8, color=TINTA)
    axs[0].set_ylabel("Registros")
    fig.suptitle("Histogramas: el escalado cambia la escala, no la forma de la distribución", x=0.01,
ha="left", fontweight="bold", fontsize=10.5)
    fig.tight_layout()
    fig.savefig(FIG / "02_histogramas_edad.png")
    plt.close(fig)

def fig_boxplots(limpio, Xt):
    fig, axs = plt.subplots(1, 2, figsize=(9, 3.1), gridspec_kw={"width_ratios": [1, 1.6]})
    ax = axs[0]
    ax.boxplot([limpio["Edad"]], widths=0.5, patch_artist=True,
                boxprops={"facecolor": "#cde2fb", "edgecolor": AZUL}, medianprops={"color": NARANJA, "lw":
2},
                whiskerprops={"color": AZUL}, capprops={"color": AZUL}, flierprops={"marker": "o",
"markersize": 3, "markeredgecolor": AZUL})
    ax.set_xticks([1], ["Edad (años)"])
    q1, q3 = limpio["Edad"].quantile([0.25, 0.75])
    lim = q3 + 1.5 * (q3 - q1)
    n_out = int((limpio["Edad"] > lim).sum())
    ax.set_title(f"Edad: {n_out} valores sobre {lim:.0f} años", loc="left", fontsize=9.5)
    ax = axs[1]
    grupos = [Xt["zscore__Edad"], Xt["ordinal__Escolaridad"], Xt["ordinal__Ingresos"], Xt["minmax__Anio"]]
    ax.boxplot(grupos, widths=0.5, patch_artist=True,
                boxprops={"facecolor": "#cde2fb", "edgecolor": AZUL}, medianprops={"color": NARANJA, "lw":
2},
                whiskerprops={"color": AZUL}, capprops={"color": AZUL}, flierprops={"marker": "o",
"markersize": 3, "markeredgecolor": AZUL})
    ax.set_xticks([1, 2, 3, 4], ["Edad\n(Z-score)", "Escolaridad\n(ordinal 0-1)", "Ingresos\n(ordinal 0-
1)", "Año\n(Min-Max)"])
    ax.set_title("Después: variables numéricas en escalas comparables", loc="left", fontsize=9.5)
    fig.tight_layout()
    fig.savefig(FIG / "03_boxplots.png")
    plt.close(fig)
    return {"limite_superior_iqr": round(float(lim), 1), "atipicos_edad": n_out}

def fig_dispersion(limpio, Xt):
    rng = np.random.default_rng(0)
    idx = rng.choice(len(limpio), 3000, replace=False)
    fig, axs = plt.subplots(1, 2, figsize=(9, 3.3))
    esc = pd.Categorical(limpio["Escolaridad"], categories=ORDEN_ESCOLARIDAD).codes
    axs[0].scatter(limpio["Edad"].to_numpy()[idx], esc[idx] + rng.uniform(-0.3, 0.3, len(idx)), s=6,
alpha=0.35, color=AZUL, edgecolors="none")

```

```

    axs[0].set_xlabel("Edad (años)")
    axs[0].set_ylabel("Nivel de escolaridad (código 0-11)")
    axs[0].set_title("Antes: escalas de 15-95 y 0-11", loc="left", fontsize=9.5)
    axs[1].scatter(Xt["zscore__Edad"].to_numpy()[idx], Xt["ordinal__Escolaridad"].to_numpy()[idx] +
rng.uniform(-0.03, 0.03, len(idx)),
                    s=6, alpha=0.35, color=AZUL, edgecolors="none")
    axs[1].set_xlabel("Edad (Z-score)")
    axs[1].set_ylabel("Escolaridad (Ordinal + Min-Max)")
    axs[1].set_title("Después: ambas en rangos comparables", loc="left", fontsize=9.5)
    r = np.corrcoef(limpio["Edad"], esc)[0, 1]
    for ax in axs:
        ax.text(0.98, 0.04, f"r de Pearson = {r:.2f}", transform=ax.transAxes, ha="right", fontsize=8,
color=TINTA)
    fig.suptitle("Dispersión Edad vs. Escolaridad (muestra de 3.000; la relación se conserva)", x=0.01,
ha="left", fontweight="bold", fontsize=10.5)
    fig.tight_layout()
    fig.savefig(FIG / "04_dispersión.png")
    plt.close(fig)
    return round(float(r), 3)

def fig_objetivo(limpio):
    vc = limpio["Lesion"].value_counts()
    vc2 = limpio["Lesion_Modelo"].value_counts()
    fig, axs = plt.subplots(1, 2, figsize=(9, 3.0))
    axs[0].bar(range(len(vc)), vc.to_numpy(), color=[AZUL if v >= m.MIN_CASOS_LESION else NARANJA for v in
vc], width=0.85)
    axs[0].set_yscale("log")
    axs[0].axhline(m.MIN_CASOS_LESION, color=TINTA2, lw=1, ls="--")
    axs[0].text(len(vc) - 1, m.MIN_CASOS_LESION * 1.2, "umbral: 30 casos", ha="right", fontsize=8,
color=TINTA)
    axs[0].set_title(f"Antes: {len(vc)} clases (naranja = menos de 30 casos)", loc="left", fontsize=9.5)
    axs[0].set_xlabel("Tipo de lesión (ordenado por frecuencia)")
    axs[0].set_ylabel("Registros (escala log)")
    axs[1].bar(range(len(vc2)), vc2.to_numpy(), color=[NARANJA if k == m.ETIQUETA_OTRAS else AZUL for k in
vc2.index], width=0.85)
    axs[1].set_yscale("log")
    axs[1].set_title(f"Después: {len(vc2)} clases (naranja = grupo 'poco frecuentes')", loc="left",
fontsize=9.5)
    axs[1].set_xlabel("Tipo de lesión (ordenado por frecuencia)")
    for ax in axs:
        ax.set_xticks([])
    fig.tight_layout()
    fig.savefig(FIG / "05_objetivo.png")
    plt.close(fig)

def fig_impacto(impacto):
    etiquetas = ["Original\nLabelEnc. + MLP", "Depurado\nOne-Hot + RL", "Línea base\nprevalencia"]
    fig, axs = plt.subplots(1, 2, figsize=(9.5, 2.9))
    for ax, k, t in [(axs[0], "top1", "Acierto en la 1ª opción"), (axs[1], "top3", "Acierto en el top 3")]:
        v = [r[k] for r in impacto]
        ax.bar(range(3), v, color=[GRIS, AZUL, "#d5ddd8"], width=0.6)
        for i, x in enumerate(v):
            ax.text(i, x + 0.01, f"{x:.1%}".replace(".", ","), ha="center", fontsize=9, color=TINTA)
        ax.set_xticks(range(3), etiquetas, fontsize=8.5)
        ax.set_ylim(0, max(v) * 1.2)
        ax.yaxis.set_major_formatter(matplotlib.ticker.PercentFormatter(1.0, decimals=0))
        ax.set_title(t, loc="left")
    fig.tight_layout()
    fig.savefig(FIG / "06_impacto_modelo.png")

```

```

plt.close(fig)

def main():
    FIG.mkdir(parents=True, exist_ok=True)
    SALIDAS.mkdir(exist_ok=True)
    crudo = pd.read_csv(m.RUTA_DATOS, sep=";", encoding="latin-1")
    crudo.columns = [c.strip() for c in crudo.columns]
    crudo_ren = crudo.rename(columns=m.COLUMNAS)

    log = []
    limpio = depurar(crudo, log)

    ct = transformador()
    X = ct.fit_transform(limpio[NOMINALES + ORDINALES + NUM_Z + NUM_MINMAX])
    Xt = pd.DataFrame(X, columns=ct.get_feature_names_out())
    y = LabelEncoder().fit_transform(limpio["Lesion_Modelo"])
    Xt["objetivo"] = y

    limpio.to_csv(SALIDAS / "obs_salud_limpio.csv", index=False, encoding="utf-8-sig")
    Xt.to_csv(SALIDAS / "obs_salud_transformado.csv", index=False, float_format="%.5g")

    bloques = {}
    for nombre in ct.get_feature_names_out():
        b = nombre.split("__")[0]
        bloques[b] = bloques.get(b, 0) + 1
    columnas_onehot = {c: int(sum(1 for n in ct.get_feature_names_out() if n.startswith(f"onehot__{c}_")))
    for c in NOMINALES}

    fig_nulos(crudo_ren, limpio)
    fig_histogramas(limpio, Xt)
    atip = fig_boxplots(limpio, Xt)
    r = fig_dispersion(limpio, Xt)
    fig_objetivo(limpio)
    impacto = impacto_en_modelo(crudo, limpio)
    fig_impacto(impacto)

    num = lambda s: {"media": round(float(s.mean()), 3), "de": round(float(s.std(ddof=0)), 3), "min":
    round(float(s.min()), 3), "max": round(float(s.max()), 3)}
    resultados = {
        "dimension_antes": list(crudo.shape),
        "dimension_limpio": list(limpio.drop(columns="Lesion_Modelo").shape),
        "dimension_transformado": list(Xt.shape),
        "estructura_antes": estructura(crudo_ren),
        "estructura_despues": estructura(limpio),
        "pasos": log,
        "bloques_transformados": bloques,
        "columnas_onehot": columnas_onehot,
        "orden_escolaridad": ORDEN_ESCOLARIDAD,
        "orden_ingresos": ORDEN_INGRESOS,
        "escalado": {
            "Edad (original)": num(limpio["Edad"]), "Edad (Z-score)": num(Xt["zscore__Edad"]),
            "Año (original)": num(limpio["Anio"]), "Año (Min-Max)": num(Xt["minmax__Anio"]),
            "Escolaridad (ordinal 0-1)": num(Xt["ordinal__Escolaridad"]), "Ingresos (ordinal 0-1)":
num(Xt["ordinal__Ingresos"]),
        },
        "atipicos": atip,
        "correlacion_edad_escolaridad": r,
        "impacto_modelo": impacto,
        "menores_18": int((limpio["Edad"] < 18).sum()),
    }

```

```
(RAIZ / "informe" / "resultados.json").write_text(json.dumps(resultados, ensure_ascii=False, indent=2),
encoding="utf-8")
print(json.dumps({k: resultados[k] for k in ["dimension_antes", "dimension_limpio",
"dimension_transformado", "impacto_modelo"]}, ensure_ascii=False))

if __name__ == "__main__":
    main()
```

A.2 modelo.py (fragmento: carga, limpieza y pipeline del modelo)

```
def cargar_datos(ruta=RUTA_DATOS) -> pd.DataFrame:
    """Lee el CSV del observatorio y normaliza textos y categorías."""
    df = pd.read_csv(ruta, sep=";", encoding="latin-1")
    df.columns = [c.strip() for c in df.columns]
    df = df.rename(columns=COLUMNAS)

    for col in df.columns:
        if col in ("Anio", "Edad"):
            continue
        df[col] = (
            df[col]
            .astype("string")
            .str.strip()
            .str.replace(r"\s+", " ", regex=True)
        )
        # "0" aparece como valor basura en algunas columnas de texto
        df.loc[df[col].isin(["0", ""]), col] = pd.NA

    # Registros idénticos en las 14 variables (incluidos edad y síntoma) se
    # tratan como notificaciones repetidas del mismo caso.
    df = df.drop_duplicates()
    df = df.dropna(subset=FEATURES + [OBJETIVO]).copy()

    frecuencias = df[OBJETIVO].value_counts()
    raras = frecuencias[frecuencias < MIN_CASOS_LESION].index
    df["Lesion_Modelo"] = df[OBJETIVO].where(
        ~df[OBJETIVO].isin(raras), ETIQUETA_OTRAS
    )
    return df.reset_index(drop=True)

def construir_pipeline() -> Pipeline:
    """One-hot para categóricas (sin orden artificial) + regresión logística.

    La versión anterior usaba LabelEncoder + MLP: eso convierte ocupaciones y
    agentes en números (0, 1, 2...) como si tuvieran orden, y el modelo
    rendía peor que esta alternativa más simple e interpretable.
    """
    preprocesado = ColumnTransformer(
        [
            (
                "cat",
                OneHotEncoder(
                    handle_unknown="infrequent_if_exist",
                    min_frequency=MIN_FRECUENCIA_CATEGORIA,
                ),
                FEATURES_CAT,
            ),
            ("num", StandardScaler(), FEATURES_NUM),
```

```

    ]
)
return Pipeline(
    [
        ("prep", preprocesado),
        ("clf", LogisticRegression(max_iter=3000, C=0.3)),
    ]
)

```

Anexo B. Enlaces y fuentes

- Datos Abiertos Bogotá. "Enfermedades derivadas de la ocupación en Unidades de Trabajo Informal (UTI) en Bogotá D.C." Secretaría Distrital de Salud.
<https://datosabiertos.bogota.gov.co/dataset/enfermedades-derivadas-de-la-ocupacion-en-unidades-de-trabajo-informal-uti-en-bogota-d-c>
- Observatorio de Salud de Bogotá (SaluData). "Enfermedades derivadas de la ocupación en Unidades de Trabajo Informal."
<https://saludata.saludcapital.gov.co/osb/en/indicadores/enfermedades-derivadas-de-la-ocupacion/>
- SaluData. Sección Salud Laboral. <https://saludata.saludcapital.gov.co/osb/datos-de-salud/salud-laboral/>
- Repositorio del proyecto (código, datos y aplicación).
<https://github.com/CLaireMor/prediccion-riesgos-ia>
- Documentación de scikit-learn: OneHotEncoder, OrdinalEncoder, StandardScaler, MinMaxScaler y LogisticRegression.
<https://scikit-learn.org/stable/modules/preprocessing.html>